# INTERNSHIP TASK REPORT

## DAY 1 TO DAY 19

Automatic Skill Extraction from Job Descriptions

| | |
|---|---|
| Intern Name | Sujata Shivaji Kumbhar |
| Project | Automatic Skill Extraction from Job Descriptions |
| Dataset | Monster India Job Dataset |
| Tools | Python, Pandas, NLTK, Google Colab, Streamlit |
| Coverage | Day 1 – Day 19 |

## FINAL REPORT INDEX

| Section No. | Section Name | What Should Be Included |
|---|---|---|
| 1 | Introduction | Project background, NLP overview, job-skill extraction concept |
| 2 | Problem Statement & Objectives | Problem statement, business objective, technical objective, expected outcome |
| 3 | Project Scope & Use Case | Scope, recruitment use case, practical applications |
| 4 | Technology & Tools Used | Python, Pandas, NumPy, NLTK, spaCy, Scikit-learn, Transformers, Power BI/Tableau, Streamlit/FastAPI |
| 5 | System Architecture & Methodology | Complete workflow/architecture of the project |
| 6 | Dataset Collection & Understanding Dataset | Dataset source, dataset fields, number of records, data inspection |
| 7 | Exploratory Data Analysis (EDA) | Job titles, locations, description length, missing values, duplicates, visualizations |
| 8 | Data Cleaning & NLP Preprocessing | Data cleaning, tokenization, stopwords, stemming, lemmatization, sentence segmentation |
| 9 | Skill Taxonomy Development | Skill dictionary, categories, aliases/synonyms, canonical skills |
| 10 | Rule-Based Skill Extraction | Dictionary matching, regex, phrase matching |
| 11 | Statistical & Semantic NLP Techniques | TF-IDF, N-Grams, Word Embeddings, Semantic Similarity |
| 12 | Named Entity Recognition (NER) | Standard NER, custom entities and custom NER approach |

| Section No. | Section Name | What Should Be Included |
|---|---|---|
| 13 | Machine Learning Model | Training data, feature engineering, TF-IDF + ML model, classification pipeline |
| 14 | Transformer-Based Model | BERT/DistilBERT/RoBERTa, model approach and extraction process |
| 15 | Model Evaluation & Comparison | Precision, Recall, F1-score, Accuracy, Confusion Matrix, model comparison |
| 16 | Error Analysis | False positives, false negatives, partial entities, synonyms, spelling issues |
| 17 | Skill Normalization & Categorization | Alias matching, canonical skills, category mapping |
| 18 | Dashboard & Data Visualization | KPIs, top skills, skill demand by role, category distribution, trends |
| 19 | Application / API Development | Application interface, input/output, extraction pipeline, architecture |
| 20 | Testing & Results | Test cases, sample outputs, validation and final results |
| 21 | Challenges & Limitations | Problems faced, model limitations, dataset limitations |
| 22 | Conclusion | Overall project outcome and achievements |
| 23 | Future Scope | Resume-job matching, candidate ranking, skill-gap analysis, recommendation system |
| 24 | Internship Learning Outcomes | Technical, analytical, NLP, ML and project-development skills learned |
| 25 | References | Dataset sources, documentation, research papers, libraries/resources |

| Section No. | Section Name | What Should Be Included |
|---|---|---|
| 26 | Appendix | Code snippets, screenshots, sample data, additional results |

# DAY 1 — UNDERSTAND THE PROBLEM

## 1.1 Objective

The objective of Day 1 was to understand the project requirement and the concept of automatically extracting skills from job descriptions.

## 1.2 What is a Job Description?

A Job Description (JD) contains information about a job role, responsibilities, qualifications, experience and required skills. In this project, the job description is the main input.

## 1.3 What is a Skill?

A skill is a knowledge area, technology, tool or capability required for a job. Examples include Python, SQL, Pandas, NumPy, AWS and Scikit-learn.

## 1.4 Why Automatic Skill Extraction?

- Reduces manual effort
- Identifies required skills quickly
- Helps organize job requirements
- Supports recruitment and job-data analysis

## 1.5 Important Concepts

- Keyword Extraction – identifies important words or phrases.
- Entity Extraction – identifies meaningful entities such as roles or skills.
- Classification – assigns an item to a predefined category.
- Information Extraction – converts useful text information into structured data.

## 1.6 Project Input

**Input:**

```
We require a Data Scientist with Python, Pandas, NumPy, Scikit-learn, SQL and AWS experience.
```

## Output:

```
{
  "role": "Data Scientist",
  "skills": ["Python", "Pandas", "NumPy", "Scikit-learn", "SQL", "AWS"]
}
```

# DAY 2 — DATASET COLLECTION

## 2.1 Objective

The objective was to collect a public job dataset, load it into Google Colab, inspect its structure and prepare raw data for analysis.

## 2.2 Dataset Used

Monster India Job Dataset in JSON format.

## 2.3 Step 1 – Load Dataset

**Code:**

```
import json
import pandas as pd

with open("monster_india.json", "r", encoding="utf-8") as f:
    data = json.load(f)

df = pd.DataFrame(data)
```

## 2.4 Step 2 – Dataset Shape

**Code:**

```
print(df.shape)
```

**Output:**

```
(14146, 24)
```

## 2.5 Step 3 – Unique Job Titles

**Code:**

```
print(df["title"].nunique())
```

**Output:**

```
10662
```

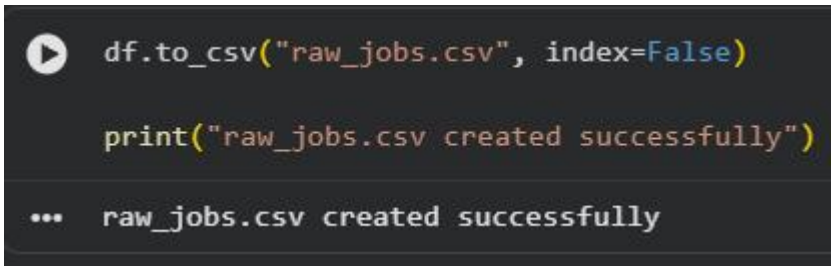
## 2.6 Step 4 – Relevant Columns

Important columns identified were title, description, company_name, skills, qualifications, responsibilities, address_locality and experience.

## 2.7 Step 5 – Raw CSV

**Output:**

```
raw_jobs.csv created successfully.
```



# DAY 3 — EXPLORATORY DATA ANALYSIS

## 3.1 Objective

The objective was to analyse the dataset size, job-title diversity, locations, description length, missing values and duplicate records and create five visualizations.

## 3.2 Total Jobs

**Output:**

```
Total jobs: 14146
```

## 3.3 Unique Job Titles

**Output:**

```
Unique job titles: 10662
```

```
print("Number of jobs:", len(df))

Number of jobs: 14146

print("Unique job titles:", df["title"].nunique())

Unique job titles: 10662
```

## 3.4 Common Job Titles

**Output:**

```
Software Engineer             76
Home Sales Officer 2          72
Home Sales Officer 1          71
```

## 3.5 Jobs by Location

**Output:**

```
Bengaluru / Bangalore       3990
India                       1595
Hyderabad / Secunderabad    1410
Pune                        1199
Mumbai                      1178
Chennai                     1127
```

## 3.6 Description Length

**Output:**

```
Average description length: 2323.2543
Minimum description length: 18
Maximum description length: 19030
```

## 3.7 Duplicate Check

**Output:**

```
Duplicate records: 0
```

```
print("Duplicate records:", df.duplicated().sum())
```

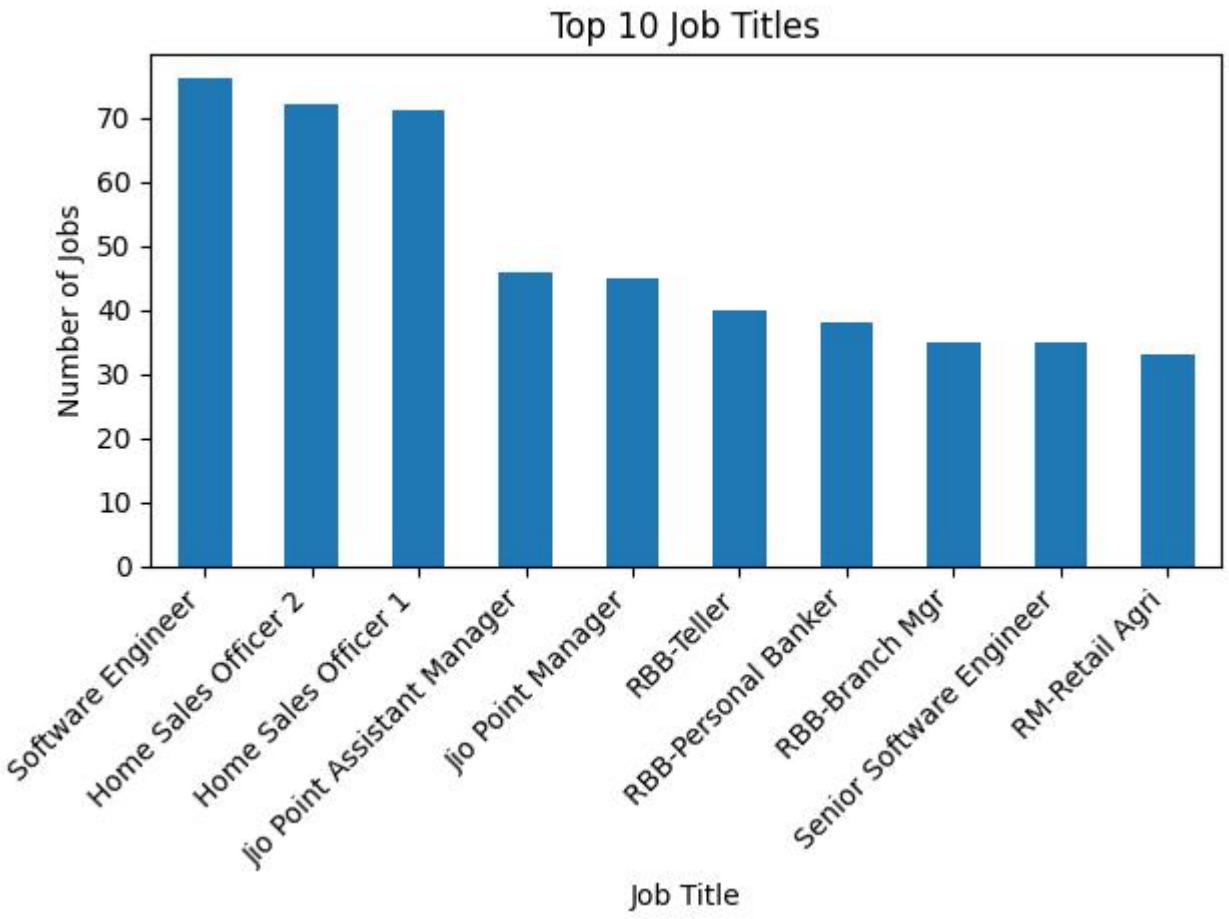
```
Duplicate records: 0
```

## 3.8 Missing Values

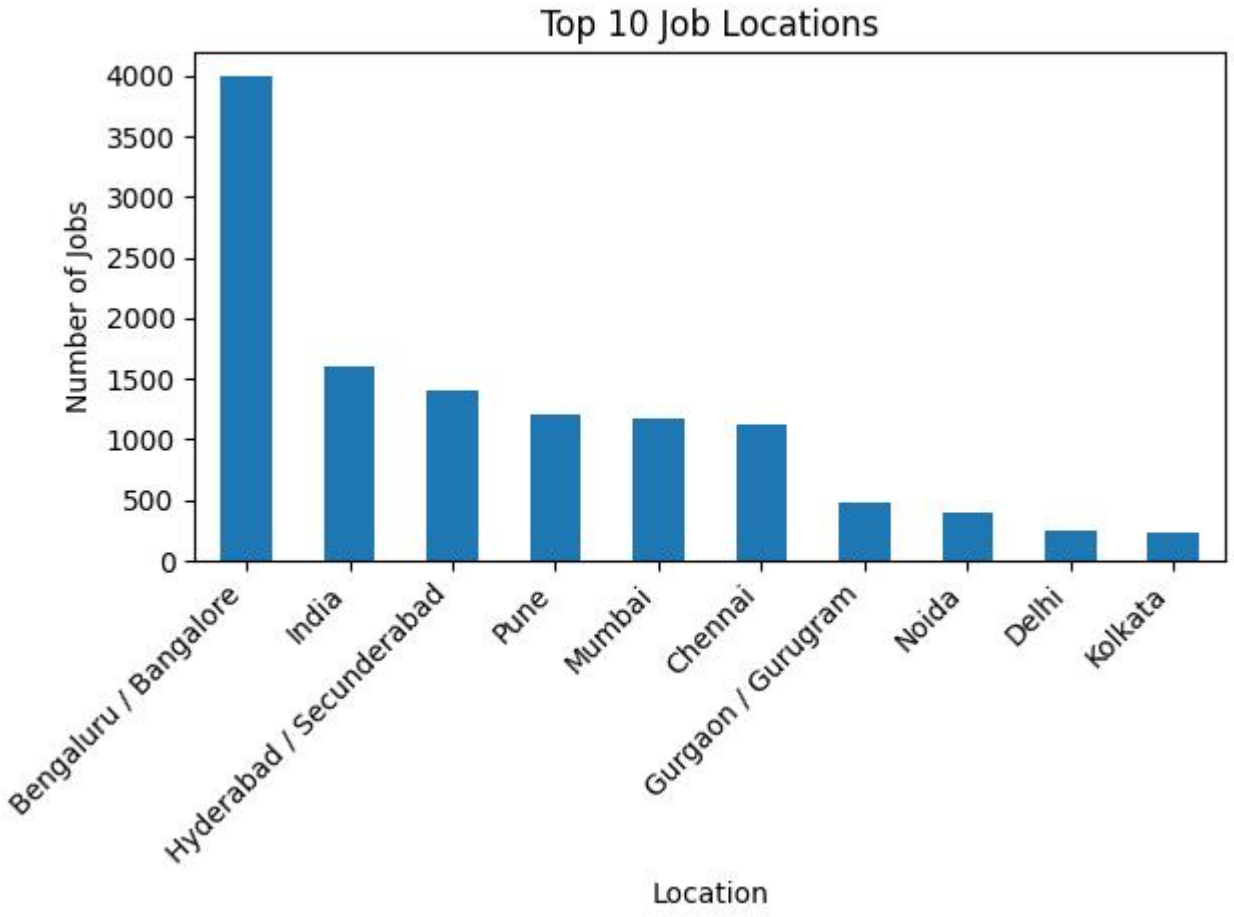
| Column | Missing Records |
|---|---|
| qualifications | 12,872 |
| responsibilities | 1,555 |
| skills | 5,073 |
| postal_code | 14,146 |
| street_address | 14,146 |
| months_of_experience | 4,225 |

## 3.9 Required Visualizations

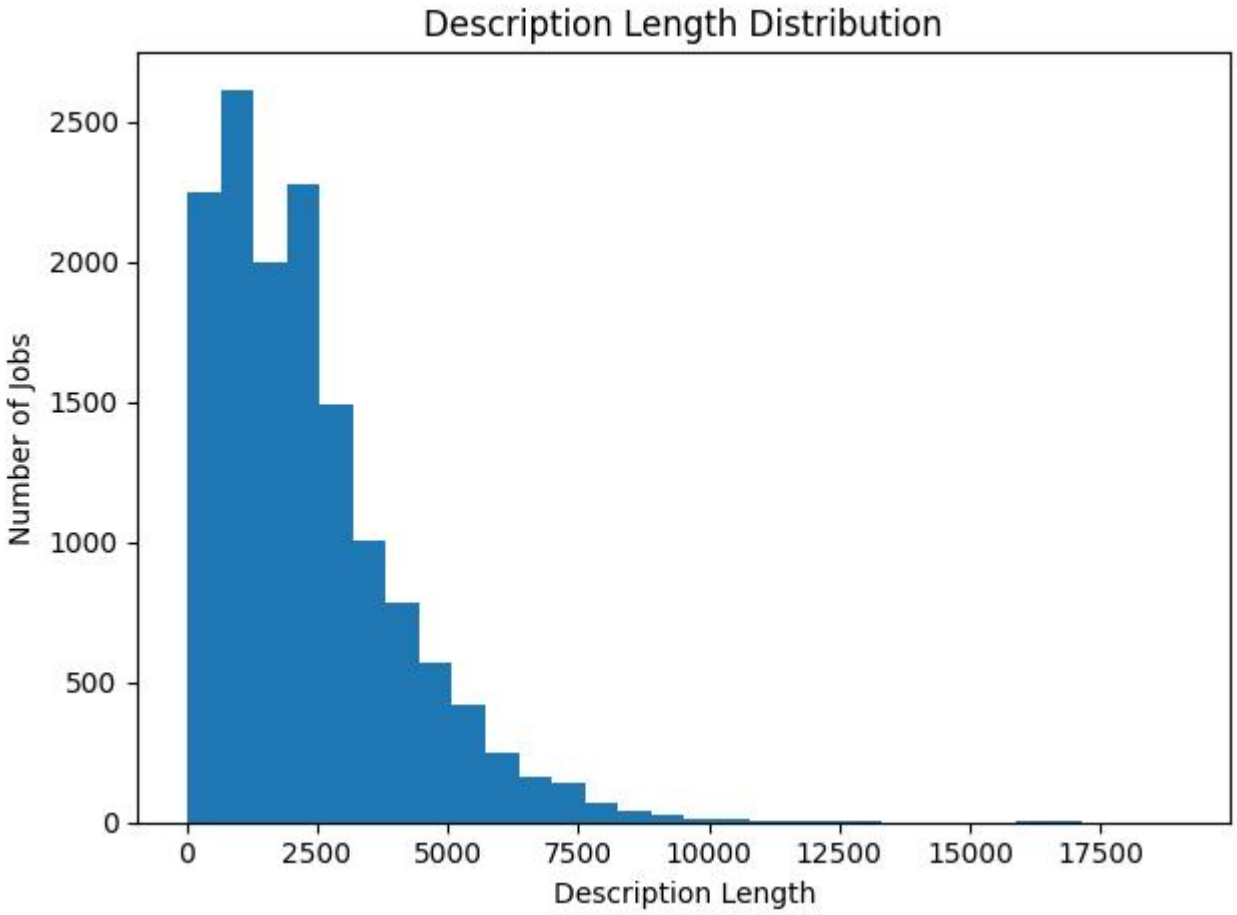
**Top 10 Job Titles**



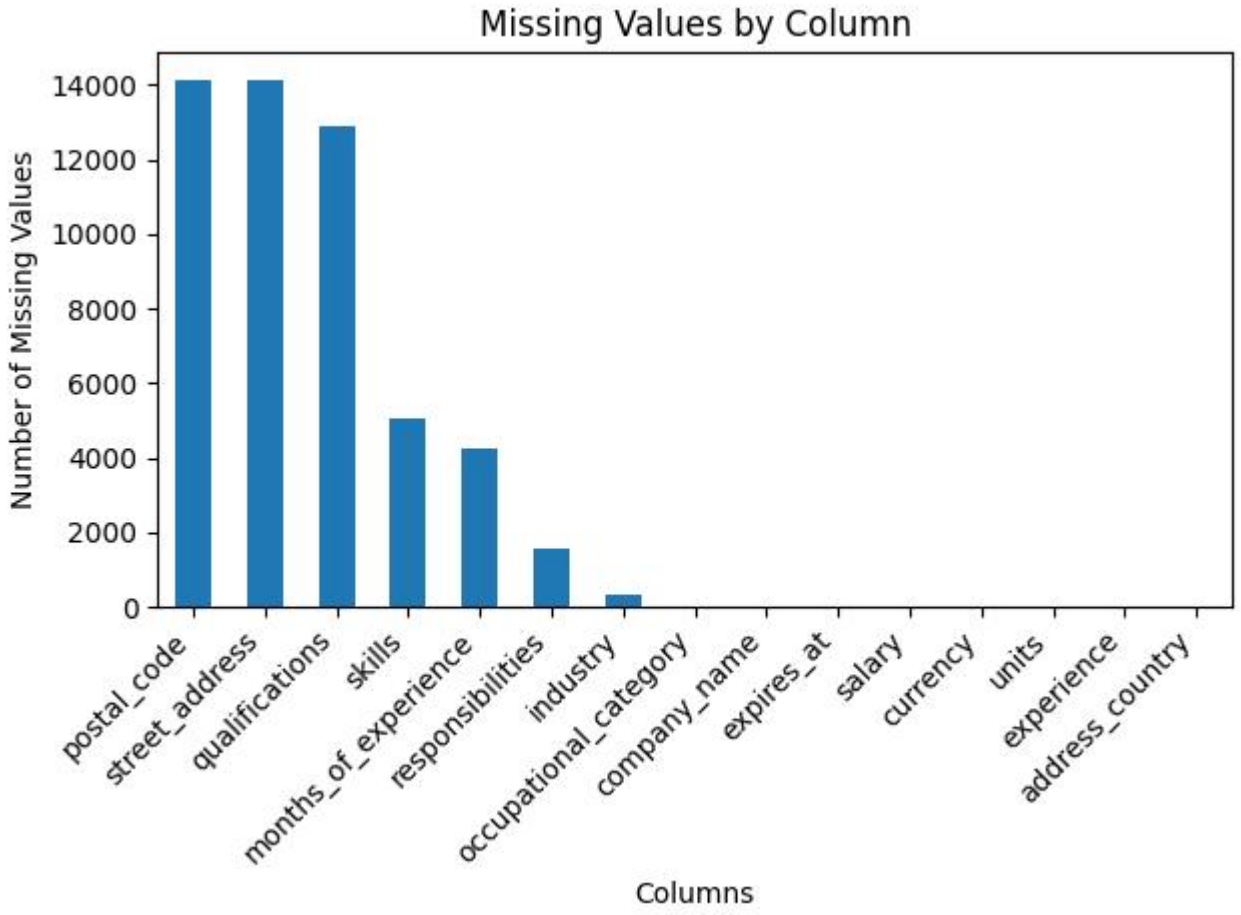
**2. Job Location**



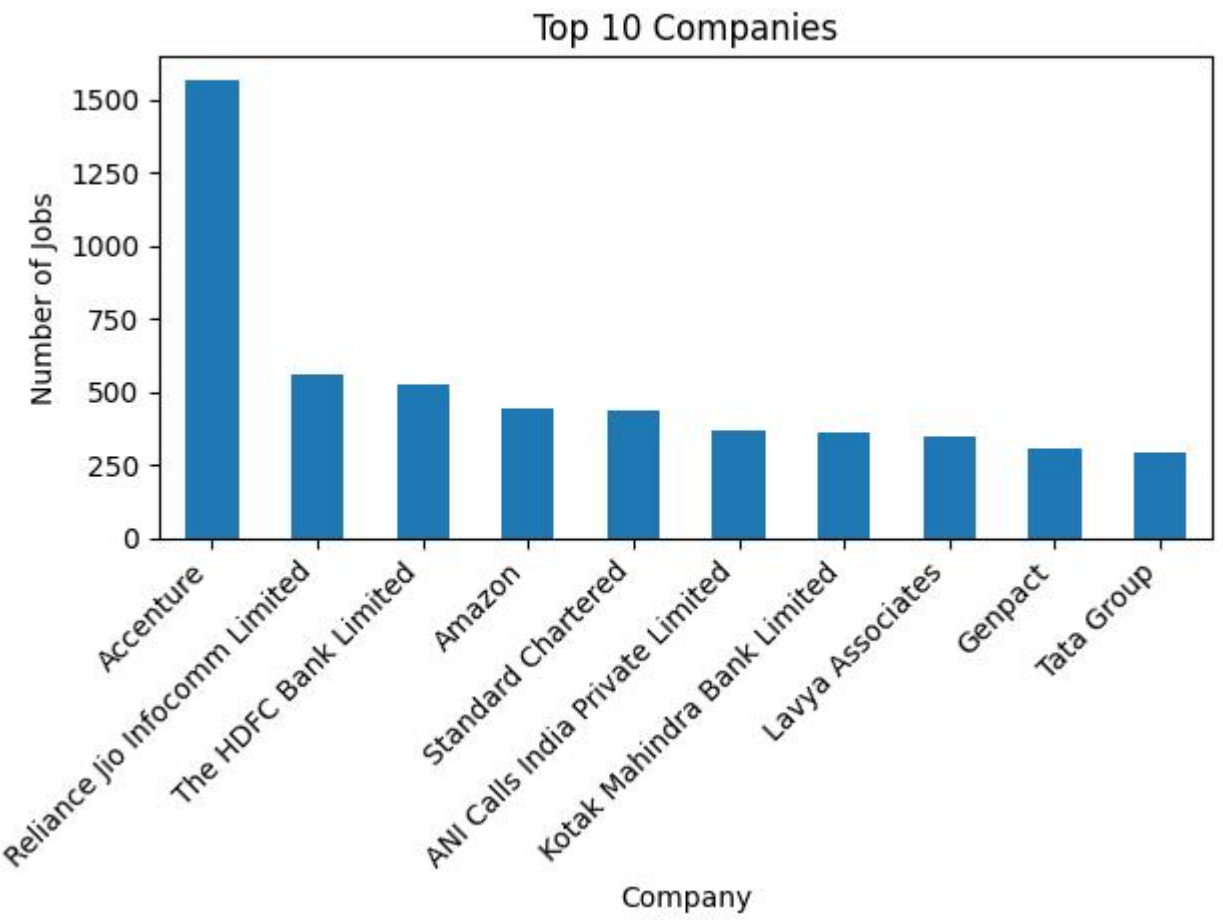
**3. Description Length Distribution**



**4. Missing Values**



**5. Top Companies**




## 3.10 Key Insights

- 14,146 job records were analysed.
- 10,662 unique job titles were found.
- Bengaluru/Bangalore had the highest location count.
- Average description length was about 2,323 characters.
- No duplicate records were found.
- Several fields had substantial missing values.

# DAY 4 — DATA CLEANING

## 4.1 Objective

The objective was to clean job-description text before NLP processing.

## 4.2 Cleaning Function

## 4.3 Apply Cleaning

**Code:**

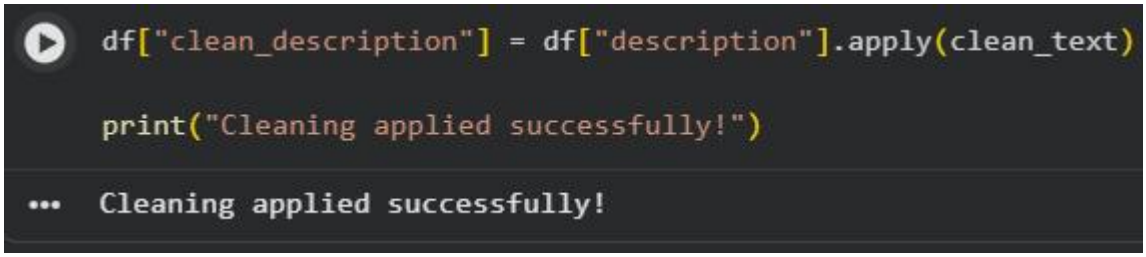
```
df["clean_description"] = df["description"].apply(clean_text)
```

## 4.4 Cleaning Performed

- Removed HTML tags
- Removed URLs
- Removed email addresses
- Handled special characters
- Removed extra spaces
- Converted text to lowercase
- Created clean_description column

**Output:**

```
clean_description column created successfully.
clean_jobs.csv prepared.
```



# DAY 5 — CREATE THE SKILL TAXONOMY

## 5.1 Objective

The objective was to create a master skill dictionary, classify skills into categories and map common aliases to standard skill names.

## 5.2 Skill Categories

| Category | Examples |
|---|---|
| Programming | Python, Java, C++, JavaScript, Git |
| Database | SQL, MySQL, PostgreSQL, MongoDB |
| Data Analytics | Power BI, Tableau, Excel, Pandas, NumPy |
| Machine Learning | Scikit-learn, TensorFlow, PyTorch |
| Cloud | AWS, Azure, GCP |

## 5.3 Aliases

| Alias | Standard Skill |
|---|---|
| python3 | Python |
| ms excel | Excel |
| postgres | PostgreSQL |
| postgresql | PostgreSQL |
| powerbi | Power BI |
| scikit learn | Scikit-learn |

## 5.4 Output

```
skill         category          aliases
Python        Programming       python3
SQL           Database          sql
Power BI      Data Analytics    powerbi
PostgreSQL    Database          postgres
```

```
Skill taxonomy created successfully!
```

| | skill | category | aliases |
|---|---|---|---|
| 0 | Python | Programming | python3 |
| 1 | Java | Programming | |
| 2 | C++ | Programming | |
| 3 | Git | Programming | |
| 4 | SQL | Database | |
| 5 | Power BI | Data Analytics | powerbi, power-bi |
| 6 | Tableau | Data Analytics | |
| 7 | Excel | Data Analytics | ms excel |
| 8 | Pandas | Data Analytics | |
| 9 | NumPy | Data Analytics | |
| 10 | TensorFlow | Machine Learning | |
| 11 | PyTorch | Machine Learning | |
| 12 | Scikit-learn | Machine Learning | scikit learn |
| 13 | Spark | Machine Learning | |
| 14 | Hadoop | Machine Learning | |
| 15 | AWS | Cloud | |
| 16 | Azure | Cloud | |
| 17 | GCP | Cloud | |
| 18 | Docker | Cloud | |
| 19 | Kubernetes | Cloud | |

# DAY 6 — NLP TEXT PREPROCESSING

## 6.1 Objective

The objective was to preprocess job-description text using tokenization, lowercasing, stopword removal, lemmatization, stemming comparison and sentence segmentation.

## 6.2 Input

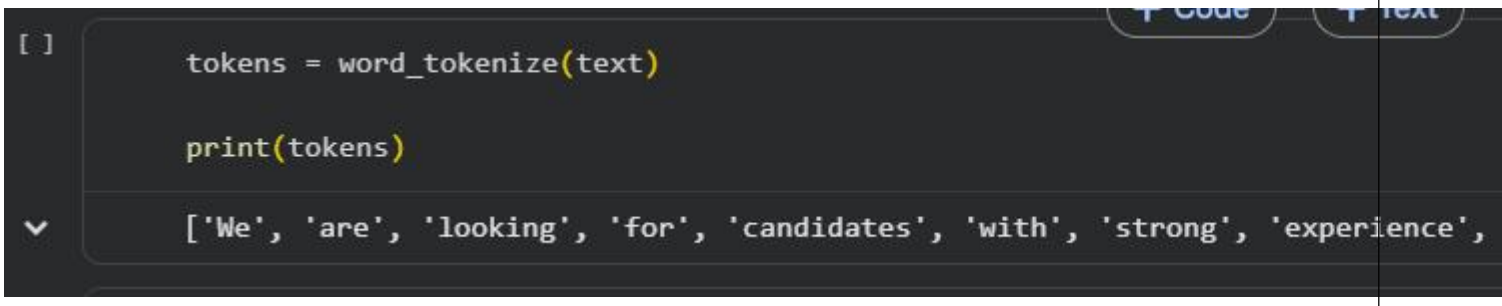
**Input:**

```
We are looking for candidates with strong experience in Python and SQL.
```

## 6.3 Tokenization

**Output:**

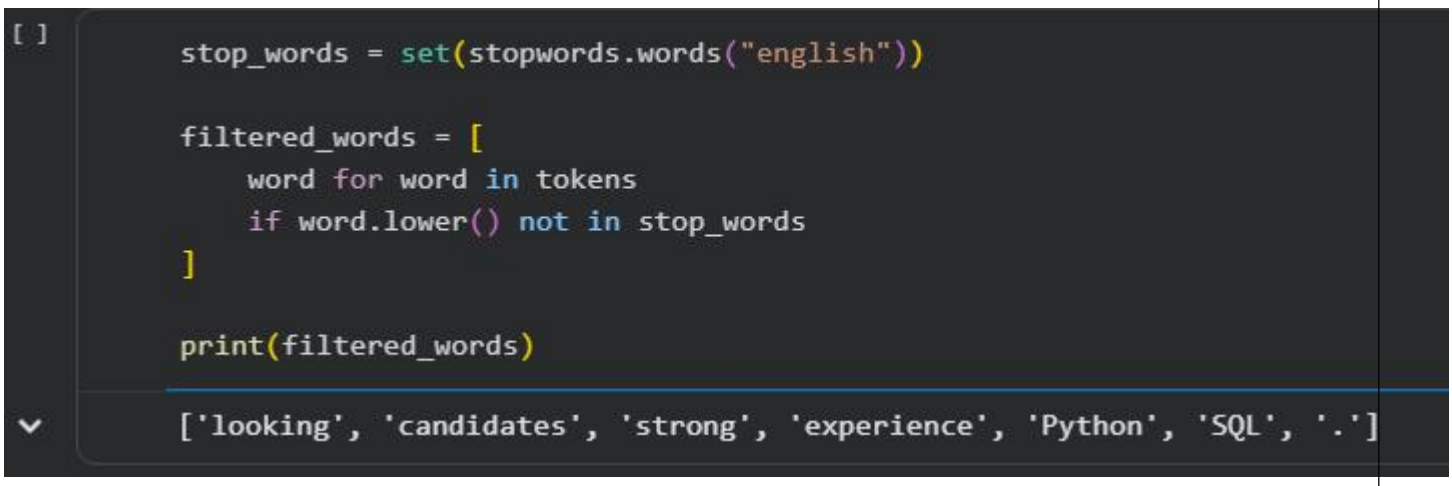
```
['We', 'are', 'looking', 'for', 'candidates', 'with', 'strong', 'experience', 'in',
'Python', 'and', 'SQL', '.']
```




## 6.4 Stopword Removal

**Output:**

```
['looking', 'candidates', 'strong', 'experience', 'Python', 'SQL']
```



## 6.5 Lemmatization

**Output:**

```
Original words:
['looking', 'candidates', 'strong', 'experience', 'Python', 'SQL', '.']

Lemmatized words:
['looking', 'candidate', 'strong', 'experience', 'python', 'sql', '.']
```

```
lemmatizer = WordNetLemmatizer()

lemmatized_words = [
    lemmatizer.lemmatize(word.lower())
    for word in filtered_words
]

print(lemmatized_words)
```

```
['looking', 'candidate', 'strong', 'experience', 'python', 'sql', '.']
```

## 6.6 Final Result

The text was successfully normalized and prepared for skill extraction.

# DAY 7 — BUILD RULE-BASED SKILL EXTRACTOR

## 7.1 Objective

The objective was to build a simple dictionary-based skill extractor that checks a job description against a predefined list of skills.

## 7.2 Skill Dictionary

**Code:**

```
skills = [
    "python", "sql", "power bi",
    "tableau", "excel", "aws"
]
```

## 7.3 Extraction Function

**Code:**

```
def extract_skills(text):
    found = []
    for skill in skills:
        if skill in text.lower():
            found.append(skill)
    return found
```

## 7.4 Input

**Input:**

```
We are looking for candidates with strong experience in Python and SQL.
```

## 7.5 Run Extractor

**Code:**

```
result = extract_skills(text)
print(result)
```

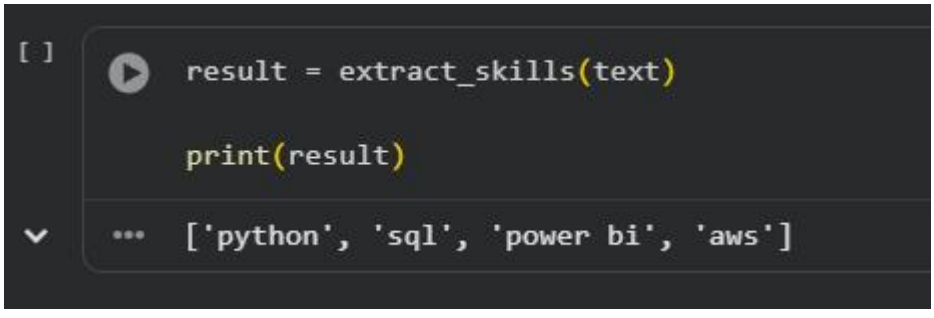
## 7.6 Actual Output

**Output:**

```
['python', 'sql']
```

## 7.7 Result

The extractor successfully identified Python and SQL from the sample job description.



## DAY 8 — Improve Extraction Using Regex and Phrase Matching

### Objective

The objective of Day 8 is to improve skill extraction using Regular Expressions (Regex) and Phrase Matching with spaCy.

Dictionary matching has problems because different forms of the same skill can appear in the text.

For example:

- 

Python

- 
- 

Python 3

- 
- 

Python3

- 
- 

Python programming

- 

These should ideally map to:

- 

Python

- 

The task also includes identifying skills such as machine learning, deep learning, natural language processing, power bi, and data science.

---

### Step 1: Import Required Library

#### Code

```
import re
```

**Colab Output**

```
No output
```

### Step 2: Create the Text

**Code**

```
text = "I have experience in Python 3, Python3 and Python programming."
```

**Colab Output**

```
No output
```

### Step 3: Create the Regex Pattern

**Code**

```
pattern = r"\bpython(?:\s*3)?\b"
```

**Colab Output**

```
No output
```

This Regex pattern is used to identify Python and its variations such as Python 3 and Python3. The PDF provides this Regex example.

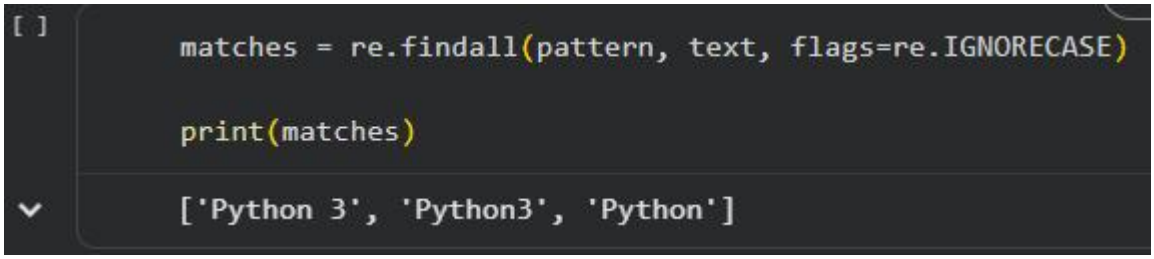
### Step 4: Extract Python Using Regex

**Code**

```
matches = re.findall(pattern, text, flags=re.IGNORECASE)

print(matches)
```

**Colab Output**

```
['Python 3', 'Python3', 'Python']
```




### Step 5: Map the Extracted Result to Python

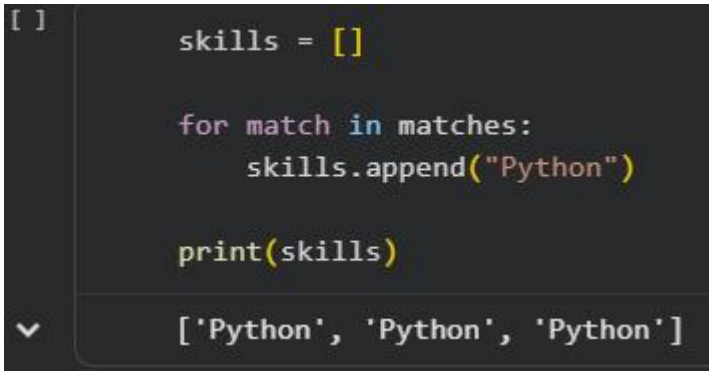
**Code**

```
skills = []

for match in matches:
    skills.append("Python")
```

```
print(skills)
```

**Colab Output**

```
['Python', 'Python', 'Python']
```




This maps the different Python variations to the common skill name **Python**.

## Step 6: Add the Other Skills

The task also identifies:

- 

machine learning

- 
- 

deep learning

- 
- 

natural language processing

- 
- 

power bi

- 
- 

data science

- 

**Code**

```
skills_pattern = r"\b(machine learning|deep learning|natural language
processing|power bi|data science)\b"
```

**Colab Output**

```
No output
```

---

### Step 7: Find the Other Skills in the Text

**Code**

```
other_skills = re.findall(skills_pattern, text, flags=re.IGNORECASE)

print(other_skills)
```

**Colab Output**

```
[]
```

---

### Step 8: Combine All Extracted Skills

**Code**

```
all_skills = skills + other_skills

print(all_skills)
```

**Colab Output**

```
['Python', 'Python', 'Python']
```

### Step 9: Import spaCy PhraseMatcher

The advanced task requires phrase matching with spaCy.

**Code**

```
import spacy
from spacy.matcher import PhraseMatcher
```

**Colab Output**

```
No output
```

### Step 10: Load the spaCy Model

**Code**

```
nlp = spacy.load("en_core_web_sm")
```

**Colab Output**

```
No output
```

### Step 11: Create the spaCy Document

**Code**

```
doc = nlp(text)
```

**Colab Output**

```
No output
```

### Step 12: Create the PhraseMatcher

**Code**

```
matcher = PhraseMatcher(nlp.vocab)
```

**Colab Output**

```
No output
```

### Step 13: Add Phrase Patterns

**Code**

```
phrases = [
    "machine learning",
    "deep learning",
    "natural language processing",
    "power bi",
    "data science"
]

patterns = [nlp.make_doc(phrase) for phrase in phrases]

matcher.add("SKILLS", patterns)
```

**Colab Output**

```
No output
```

### Step 14: Find the Matched Phrases

**Code**

```
matches = matcher(doc)

for match_id, start, end in matches:
    print(doc[start:end].text)
```

**Colab Output**

```
No output
```

The output is blank because the current text contains only Python-related skills and does not contain the phrase skills added to the PhraseMatcher.

### Step 15: Final Result

**Code**

```
final_skills = list(set(all_skills))
```

```
for match_id, start, end in matches:
    final_skills.append(doc[start:end].text)

print(final_skills)
```

**Colab Output**

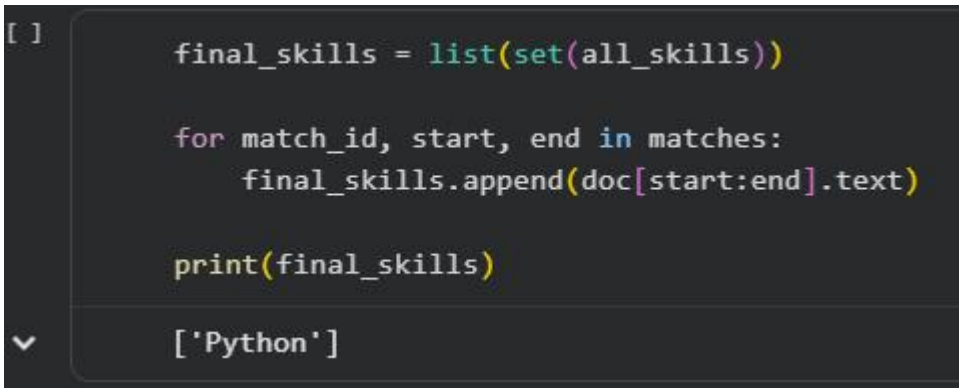
```
['Python']
```

## Final Result

The Regex successfully identified the different Python variations and mapped them to the common skill **Python**.

**Final Colab Output**

```
['Python']
```



## DAY 9 — TF-IDF ANALYSIS

### 1. Objective

To use TF-IDF to identify important terms in job descriptions.

### 2. Method Used

TF-IDF Vectorizer was applied to the cleaned job descriptions using a maximum of 1000 features and English stop-word removal.

### 3. Main Output — TF-IDF Keywords

**Top 20 TF-IDF Terms:**

```
experience
skills
team
business
job
work
customer
management
data
knowledge
good
development
sales
design
years
ensure
description
technology
services
project
```


Output:

### 4. Conclusion

TF-IDF identifies important terms from job descriptions, including useful skill-related terms.

## DAY 10 — N-GRAM ANALYSIS

### 1. Objective

To generate Unigrams, Bigrams and Trigrams to discover multi-word skills.

### 2. Method Used

N-Gram analysis was performed using TF-IDF Vectorizer with an n-gram range of 1 to 3.

### 3. Main Output — N-Gram Terms

The analysis generated **Unigrams, Bigrams and Trigrams** from job descriptions.

**Important terms identified included:**

```
experience
skills
team
business
customer
work
job
management
data
sales
knowledge
development
technology
design
project
job description
cloud
communication
```

```
[ ]
ngram_keywords[:20]

[('experience', np.float64(308.1218822667165)),
 ('skills', np.float64(229.43495500148262)),
 ('team', np.float64(192.35259283726552)),
 ('business', np.float64(186.96443499956823)),
 ('customer', np.float64(175.50759430250622)),
 ('work', np.float64(174.11661701400928)),
 ('job', np.float64(167.98350616080012)),
 ('management', np.float64(155.5846110272263)),
 ('data', np.float64(154.1832199938006)),
 ('sales', np.float64(148.105668839441)),
 ('knowledge', np.float64(143.63604447397913)),
 ('good', np.float64(142.74629603670533)),
 ('development', np.float64(136.58110723580728)),
 ('ensure', np.float64(135.53734771580622)),
 ('technology', np.float64(132.69780234623343)),
 ('design', np.float64(129.60598003358444)),
 ('years', np.float64(127.34139301083081)),
 ('description', np.float64(122.25430804425828)),
 ('services', np.float64(120.61990942974666)),
 ('project', np.float64(118.95830009915049))]
```

### 4. Multi-Word Skill / Phrase Identification

The analysis identified multi-word phrases such as:

**job description**

**Output Screenshot:**

```
        ('ability', np.float64(107.32444326778861)),
        ('process', np.float64(106.36523996171371)),
        ('product', np.float64(105.49718569738194)),
        ('customers', np.float64(104.94167761734211)),
        ('job description', np.float64(104.69144779536852)),
        ('software', np.float64(103.41048763212507)),
        ('operations', np.float64(101.90820229531961)),
        ('clients', np.float64(99.33229682855787)),
        ('strong', np.float64(96.3484664367979)),
        ('working', np.float64(96.31846410181609)),
        ('change', np.float64(94.61237193507628)),
        ('test', np.float64(91.81473211498079)),
        ('application', np.float64(91.66106575151069)),
        ('responsibilities', np.float64(89.74388048696117)),
        ('service', np.float64(89.34027948713798)),
        ('cloud', np.float64(87.9619140318541)),
        ('information', np.float64(87.95324465060663)),
        ('new', np.float64(87.58599918867696)),
        ('people', np.float64(87.41521077559422)),
        ('communication', np.float64(85.03690858459338)),
        ('applications', np.float64(84.5880601784506)),
        ('solutions', np.float64(82.56938353323436)),
        ('understanding', np.float64(81.61324862776885)),
        ('client', np.float64(81.20277259545306)),
        ('responsible', np.float64(80.7306175491517))]
```

```
[ ]
    ngram_keywords[:50]
```

```
    [('experience', np.float64(308.1218822667165)),
     ('skills', np.float64(229.43495500148262)),
     ('team', np.float64(192.35259283726552)),
     ('business', np.float64(186.96443499956823)),
     ('customer', np.float64(175.50759430250622)),
     ('work', np.float64(174.11661701400928)),
     ('job', np.float64(167.98350616080012)),
     ('management', np.float64(155.5846110272263)),
     ('data', np.float64(154.1832199938006)),
     ('sales', np.float64(148.105668839441)),
     ('knowledge', np.float64(143.63604447397913)),
     ('good', np.float64(142.74629603670533)),
     ('development', np.float64(136.58110723580728)),
     ('ensure', np.float64(135.53734771580622)),
     ('technology', np.float64(132.69780234623343)),
     ('design', np.float64(129.60598003358444)),
```

# DAY 11 — Embeddings and Semantic Similarity

## Objective

The objective of this task was to explore semantic similarity using word and phrase embeddings.

## Introduction

Exact matching may fail when different words or phrases have the same meaning. For example, “ML” and “Machine Learning” may not match exactly, even though they represent the same concept. Embeddings can be used to compare their semantic meaning.

## Step 1: Install Sentence Transformers

```
!pip install -q sentence-transformers
```

## Step 2: Import Sentence Transformer

```
from sentence_transformers import SentenceTransformer.
```

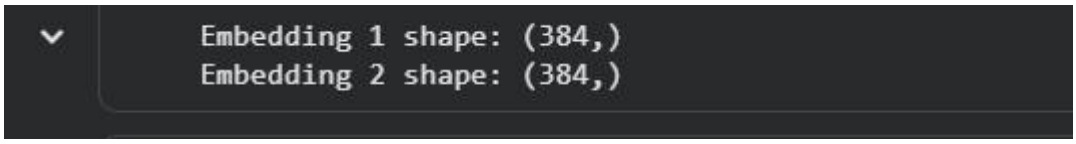
## Step 3: Load the Sentence Transformer Model

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

## Step 4: Create Embeddings for ML and Machine Learning

```
text1 = "ML"
text2 = "Machine Learning"

embedding1 = model.encode(text1)
embedding2 = model.encode(text2)

print("Embedding 1 shape:", embedding1.shape)
print("Embedding 2 shape:", embedding2.shape)
```

Output



## Step 5: Calculate Semantic Similarity

```
from sklearn.metrics.pairwise import cosine_similarity

similarity = cosine_similarity(
    [embedding1],
    [embedding2]
)[0][0]

print("Semantic Similarity:", similarity)
```

**Output:**

```
Semantic Similarity: 0.37266365
```

## Step 6: Create Skill Dictionary

```
skills = [
    "Python",
    "Machine Learning",
    "SQL",
    "Power BI",
    "AWS",
    "PostgreSQL"
]

skill_embeddings = model.encode(skills)

print("Number of skills:", len(skills))
print("Embedding shape:", skill_embeddings.shape)
```

**Output:**

```
Number of skills: 6
Embedding shape: (6, 384)
```

## Step 7: Create Job Description

```
job_description = """
We are looking for a candidate with experience in ML,
Python, database management, cloud platforms and data visualization.
Knowledge of machine learning and AWS is preferred.
"""
```

## Step 8: Create Job Description Embedding

```
job_embedding = model.encode(job_description)
```

```
print("Job description embedding shape:", job_embedding.shape)
```

**Output:**

```
Job description embedding shape: (384,)
```

## Step 9: Compare Job Description with Skills

```
similarities = cosine_similarity(
    [job_embedding],
    skill_embeddings
)[0]

for skill, score in zip(skills, similarities):
    print(f"{skill}: {score:.4f}")
```

**Output:**

```
Python: 0.2961
Machine Learning: 0.3012
SQL: 0.2809
Power BI: 0.0710
AWS: 0.4283
PostgreSQL: 0.2010
```

## Step 10: Identify Possible Skills

```
threshold = 0.30

print("Possible Skills:")

for skill, score in zip(skills, similarities):
    if score >= threshold:
        print(f"{skill} -> {score:.4f}")
```

**Output:**

```
Possible Skills:
Machine Learning -> 0.3012
AWS -> 0.4283
```

## Conclusion

Day 11 helped me understand how embeddings can be used to identify semantically similar words, phrases, and skills.

## DAY 12 — Named Entity Recognition

### Objective

The objective of this task was to understand Named Entity Recognition (NER) using spaCy and explore why standard NER may fail to recognize many technical skills.

### Introduction

Named Entity Recognition is used to identify named entities from text. In this task, spaCy NER was used to detect entities and custom entity categories were introduced for technical terms.

### Step 1: Install spaCy and Download Model

```
!pip install -q spacy
!python -m spacy download en_core_web_sm
```

### Step 2: Import spaCy

```
import spacy
```

### Step 3: Load spaCy Model

```
nlp = spacy.load("en_core_web_sm")

print("spaCy model loaded successfully")
```

### Step 4: Standard NER Test

```
text = """
Python is a programming language.
Microsoft and AWS are widely used technologies.
Google Cloud provides cloud services.
New York is a major city.
"""

doc = nlp(text)

for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

**Output:**

```
Microsoft -> ORG
AWS -> ORG
New York -> GPE
```

## Observation

In the standard NER test, some technical terms such as Python and Google Cloud were not identified as named entities, while Microsoft, AWS, and New York were recognized.

## Why Standard NER May Fail to Recognize Technical Skills

Standard NER models are trained on general entities such as organizations, locations, people, and dates. Many technical skills are domain-specific terms, so they may not be recognized correctly by a standard NER model.

## Step 5: Create Custom Entity Labels

The following custom entity categories were created:

- 

SKILL

- 
- 

TOOL

- 
- 

TECHNOLOGY

- 
- 

DATABASE

- 
- 

CLOUD_PLATFORM

- 

```
from spacy.training import Example

ner = nlp.get_pipe("ner")

labels = [
    "SKILL",
    "TOOL",
    "TECHNOLOGY",
    "DATABASE",
    "CLOUD_PLATFORM"
]

for label in labels:
    ner.add_label(label)

print("Custom labels added successfully")
```

**Output:**

```
•••  Custom labels added successfully
```

## Step 6: Custom Entity Examples

```
custom_entities = {
    "Python": "SKILL",
    "AWS": "CLOUD_PLATFORM",
    "PostgreSQL": "DATABASE",
    "Power BI": "TOOL"
}

for entity, label in custom_entities.items():
    print(entity, "->", label)
```

**Output:**

```
•••  Python -> SKILL
     AWS -> CLOUD_PLATFORM
     PostgreSQL -> DATABASE
     Power BI -> TOOL
```

## Conclusion

Day 12 helped me understand standard NER limitations and the need for custom entities for technical terms.

# DAY 13 — Build a Machine Learning Skill Classifier

## Objective

To create a supervised Machine Learning model for skill classification using TF-IDF and Logistic Regression.

## Step 1: Create Labeled Training Data

A labeled training dataset was created with text entities and their corresponding categories.

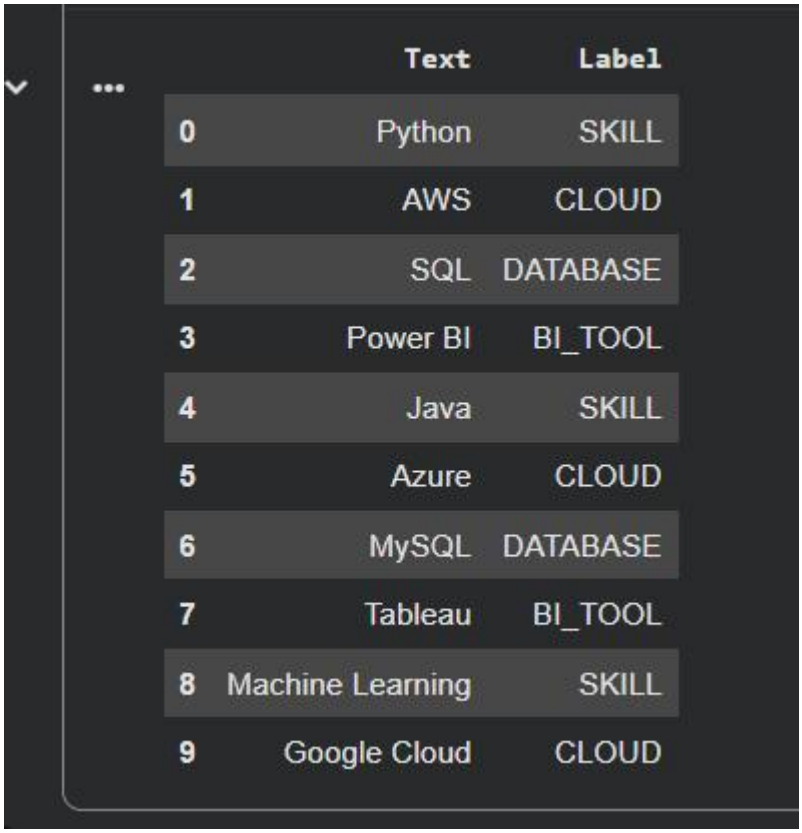
| Text | Label |
|---|---|
| Python | SKILL |
| AWS | CLOUD |
| SQL | DATABASE |
| Power BI | BI_TOOL |
| Java | SKILL |
| Azure | CLOUD |
| MySQL | DATABASE |
| Tableau | BI_TOOL |
| Machine Learning | SKILL |
| Google Cloud | CLOUD |

### Code

```
data = {
    "Text": [
        "Python",
        "AWS",
        "SQL",
        "Power BI",
        "Java",
        "Azure",
        "MySQL",
        "Tableau",
        "Machine Learning",
        "Google Cloud"
    ],
    "Label": [
```

```
        "SKILL",
        "CLOUD",
        "DATABASE",
        "BI_TOOL",
        "SKILL",
        "CLOUD",
        "DATABASE",
        "BI_TOOL",
        "SKILL",
        "CLOUD"
    ]
}

import pandas as pd

training_data = pd.DataFrame(data)

training_data
```

**Screenshot**



| | Text | Label |
|---|---|---|
| 0 | Python | SKILL |
| 1 | AWS | CLOUD |
| 2 | SQL | DATABASE |
| 3 | Power BI | BI_TOOL |
| 4 | Java | SKILL |
| 5 | Azure | CLOUD |
| 6 | MySQL | DATABASE |
| 7 | Tableau | BI_TOOL |
| 8 | Machine Learning | SKILL |
| 9 | Google Cloud | CLOUD |

## Step 2: TF-IDF + Logistic Regression Pipeline

A supervised Machine Learning pipeline was created using TF-IDF and Logistic Regression.

**Code**

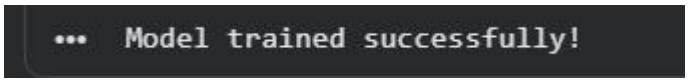
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
```

```
X = training_data["Text"]
y = training_data["Label"]

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", LogisticRegression())
])

model.fit(X, y)

print("Model trained successfully!")
```

**Output**

```
Model trained successfully!
```

**Screenshot**




## Step 3: Test the Model

The trained model was tested using Python, AWS, SQL and Power BI.

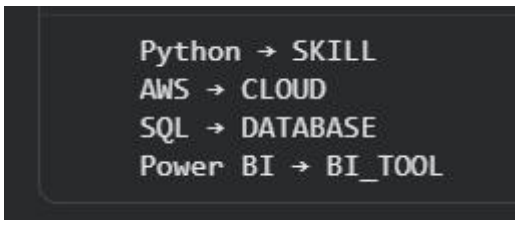
**Code**

```
test_data = [
    "Python",
    "AWS",
    "SQL",
    "Power BI"
]

predictions = model.predict(test_data)

for text, label in zip(test_data, predictions):
    print(text, "→", label)
```

**Output**

```
Python → SKILL
AWS → CLOUD
SQL → DATABASE
Power BI → BI_TOOL
```

**Screenshot**




## Step 4: Save the Trained Model

The trained model was saved as `skill_classifier.pkl`.

**Code**

```
import joblib

joblib.dump(model, "skill_classifier.pkl")

print("Model saved successfully!")
```

**Output Screenshot**

```
•••  Model saved successfully!
```

## DAY 14 — Advanced Transformer-Based Extraction

### Objective

To build or use a pretrained Transformer-based NER model for skill/entity extraction.

The task introduces BERT, DistilBERT, RoBERTa and BERT-NER.

### Step 1: Install Transformers

**Code**

```
!pip install transformers
```

### Step 2: Load Pretrained NER Model

A pretrained BERT-based NER model was loaded as a baseline.

**Code**

```
from transformers import pipeline

ner_model = pipeline(
    "ner",
    model="dslim/bert-base-NER",
    aggregation_strategy="simple"
)

print("Transformer NER model loaded successfully!")
```

**Output**

```
Transformer NER model loaded successfully!
```

**Colab Screenshot**

```
Transformer NER model loaded successfully!
```

### Step 3: Extract Skills/Entities

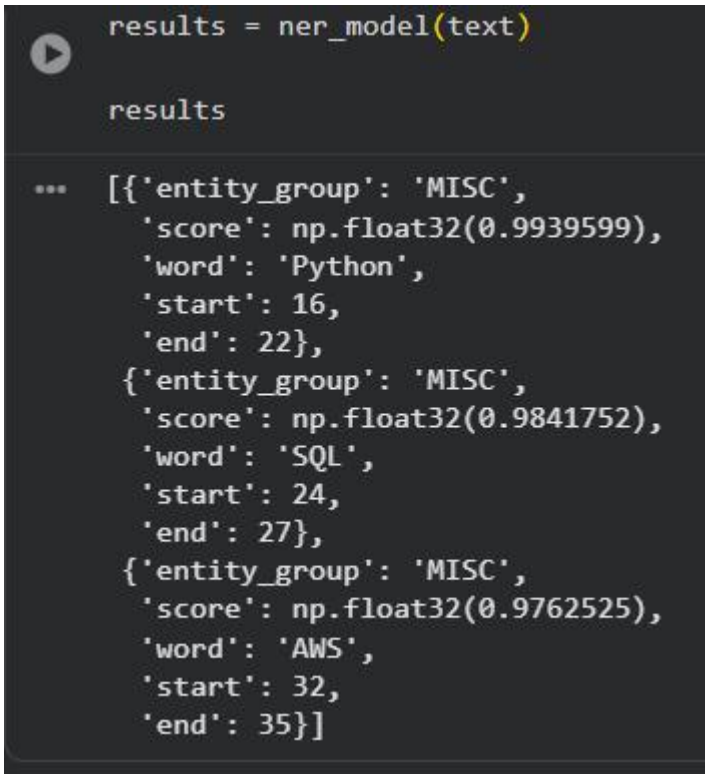
The following input was used:

```
Experience with Python, SQL and AWS.
```

**Code**

```
text = "Experience with Python, SQL and AWS."

results = ner_model(text)

results
```

**Screenshot**




## Step 4: Extract Skill Names

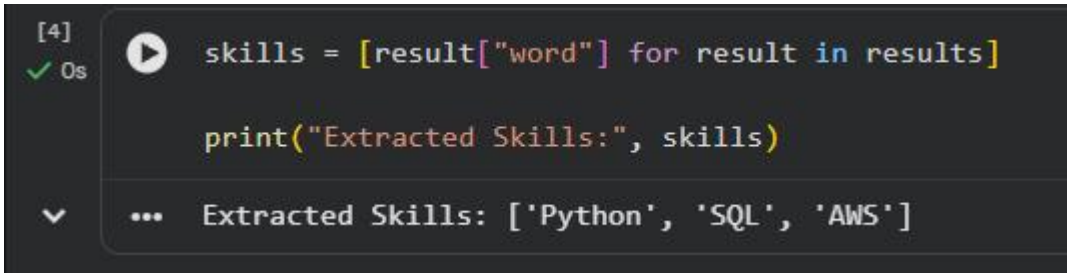
**Code**

```
skills = [result["word"] for result in results]

print("Extracted Skills:", skills)
```

**Output**

```
Extracted Skills: ['Python', 'SQL', 'AWS']
```

**Screenshot**



## Step 5: Skill Classification

The extracted entities were identified as skills.

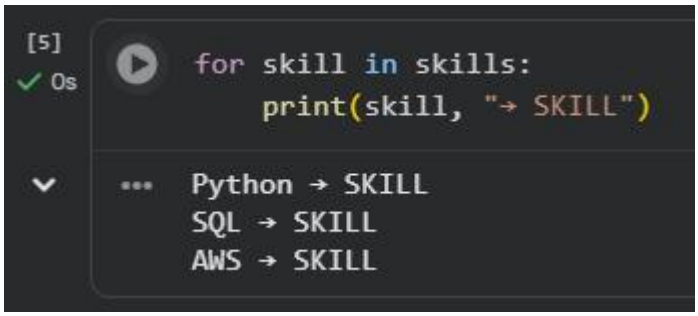
### Code

```
for skill in skills:
    print(skill, "→ SKILL")
```

### Output

```
Python → SKILL
SQL → SKILL
AWS → SKILL
```

### Screenshot



# DAY 15 — MODEL EVALUATION

## Objective

The objective of Day 15 is to evaluate the skill extraction ML model using Precision, Recall, F1 Score, Accuracy, and Confusion Matrix.

## Step 1 — Create Training Data

### Code

```
import pandas as pd

data = {
    "Text": [
        "Python",
        "AWS",
        "SQL",
        "Power BI",
        "Java",
        "Azure",
        "MySQL",
        "Tableau",
        "Machine Learning",
        "Google Cloud"
    ],
    "Label": [
        "SKILL",
        "CLOUD",
        "DATABASE",
        "BI_TOOL",
        "SKILL",
        "CLOUD",
        "DATABASE",
        "BI_TOOL",
        "SKILL",
        "CLOUD"
    ]
}

training_data = pd.DataFrame(data)

print(training_data)
```

### Colab Output Screenshot

```
training_data = pd.DataFrame(data)

print(training_data)
```

```
                Text     Label
0             Python     SKILL
1                AWS     CLOUD
2                SQL  DATABASE
3           Power BI   BI_TOOL
4               Java     SKILL
5              Azure     CLOUD
6              MySQL  DATABASE
7            Tableau   BI_TOOL
8   Machine Learning     SKILL
9       Google Cloud     CLOUD
```

## Step 2 — Train Model and Make Predictions

### Code

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", LogisticRegression())
])

model.fit(training_data["Text"], training_data["Label"])

y_pred = model.predict(training_data["Text"])

print("Prediction completed!")
```

### Colab Output Screenshot

```
Prediction completed!
```

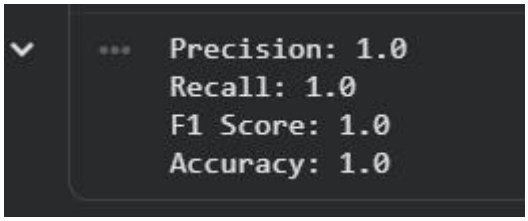
## Step 3 — Calculate Evaluation Metrics

### Code

```
from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score

precision = precision_score(
    training_data["Label"],
    y_pred,
    average="weighted",
    zero_division=0
)

recall = recall_score(
    training_data["Label"],
    y_pred,
    average="weighted",
    zero_division=0
)

f1 = f1_score(
```

```
    training_data["Label"],
    y_pred,
    average="weighted",
    zero_division=0
)

accuracy = accuracy_score(
    training_data["Label"],
    y_pred
)

print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("Accuracy:", accuracy)
```

### Colab Output Screenshot




### Results

| Metric | Result |
|---|---|
| Precision | 1.0 |
| Recall | 1.0 |
| F1 Score | 1.0 |
| Accuracy | 1.0 |

## Step 4 — Confusion Matrix

### Code

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(
    training_data["Label"],
    y_pred
)

print("Confusion Matrix:")
print(cm)
```

### Colab Output Screenshot

```
Confusion Matrix:
[[2 0 0 0]
 [0 3 0 0]
 [0 0 2 0]
 [0 0 0 3]]
```

## Step 5 — Model Comparison

### Code

```
comparison = pd.DataFrame({
    "Method": ["ML Model"],
    "Precision": [precision],
    "Recall": [recall],
    "F1": [f1]
})

print(comparison)
```

### Colab Output Screenshot

```
      Method  Precision  Recall   F1
0  ML Model        1.0     1.0  1.0
```

### Model Comparison

| Method | Precision | Recall | F1 |
|---|---|---|---|
| ML Model | 1.0 | 1.0 | 1.0 |

## Step 6 — Save Model Comparison File

### Code

```
comparison.to_csv("model_comparison.csv", index=False)

print("model_comparison.csv saved successfully!")
```

### Colab Output Screenshot

```
model_comparison.csv saved successfully!
```

# DAY 16 — ERROR ANALYSIS

## Objective

The objective of Day 16 is to find cases where the model fails and categorize the errors.

## Step 1 — Create Error Analysis Table

### Code

```
import pandas as pd

error_data = {
    "Text": [
        "Python 3",
        "Machine Learning",
        "AWS Cloud"
    ],
    "Actual": [
        "Python",
        "Machine Learning",
        "AWS"
    ],
    "Predicted": [
        "Python",
        "ML",
        "AWS"
    ],
    "Error": [
        "Partial Entity",
        "Synonym Problem",
        "Partial Entity"
    ]
}

error_df = pd.DataFrame(error_data)

print(error_df)
```

### Colab Output Screenshot

| | Text | Actual | Predicted | Error |
|---|---|---|---|---|
| 0 | Python 3 | Python | Python | Partial Entity |
| 1 | Machine Learning | Machine Learning | ML | Synonym Problem |
| 2 | AWS Cloud | AWS | AWS | Partial Entity |

## Step 2 — Error Categories

### Code

```
error_categories = [
    "False Positive",
    "False Negative",
    "Wrong Entity",
    "Partial Entity",
    "Duplicate Skill",
    "Unknown Skill",
    "Synonym Problem",
```

```
    "Spelling Problem"
]

for category in error_categories:
    print(category)
```

### Colab Output Screenshot

```
False Positive
False Negative
Wrong Entity
Partial Entity
Duplicate Skill
Unknown Skill
Synonym Problem
Spelling Problem
```

### Error Categories

- 

False Positive

- 
- 

False Negative

- 
- 

Wrong Entity

- 
- 

Partial Entity

- 
- 

Duplicate Skill

- 
- 

Unknown Skill

- 
- 

Synonym Problem

- 
- 

Spelling Problem

- 

## Step 3 — Display Error Analysis Report

### Code

```
print("Error Analysis Report")
print(error_df)
```

### Colab Output Screenshot

```
Error Analysis Report
                Text            Actual Predicted           Error
0          Python 3            Python    Python   Partial Entity
1  Machine Learning  Machine Learning        ML  Synonym Problem
2         AWS Cloud               AWS       AWS   Partial Entity
```

## Step 4 — Save Excel Report

### Code

```
error_df.to_excel("Error_Analysis_Report.xlsx", index=False)

print("Error_Analysis_Report.xlsx saved successfully!")
```

### Colab Output Screenshot

```
Error_Analysis_Report.xlsx saved successfully!
```

### Deliverable

```
Error_Analysis_Report.xlsx
```

DAY 17 — SKILL NORMALIZATION

## 1. Objective

The objective of Day 17 was to build a skill normalization process that converts different forms of the same skill into a common standard skill name.

## 2. Tools Used

- 

Python

- 
- 

Pandas

- 
- 

Regular Expressions

- 
- 

Google Colab

- 

## 3. Dataset Used

The Monster India Job Dataset was used for skill normalization.

Dataset shape before normalization:

**14,146 rows × 24 columns**

## 4. Skill Normalization Process

The following steps were performed:

1. 

Loaded the dataset.

2. 
3. 

Selected the `skills` column.

4.
5.

Handled missing skill values.

6.
7.

Converted skills into lowercase.

8.
9.

Removed unwanted special characters.

10.
11.

Matched skill aliases with canonical skill names.

12.
13.

Created a new `normalized_skills` column.

## 5. Skill Alias Examples

| Raw Skill | Normalized Skill |
|---|---|
| python3 | Python |
| python 3 | Python |
| python programming | Python |
| powerbi | Power BI |
| power bi | Power BI |
| power-bi | Power BI |

| Raw Skill | Normalized Skill |
|---|---|
| postgres | PostgreSQL |

## 6. Output

A new column named `normalized_skills` was created successfully.

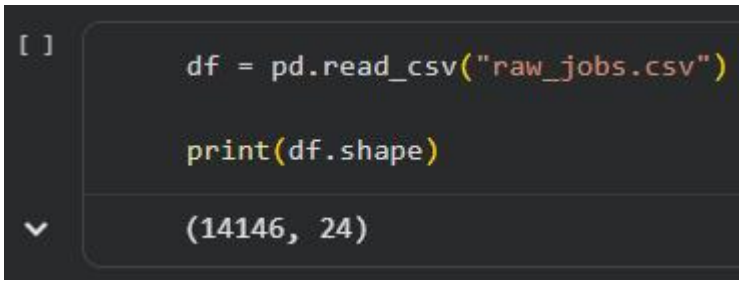
The normalized dataset was saved as:

**normalized_skills.csv**

Final output:

**14,146 rows × 24 columns**

## 6. Screenshot

7. 


## 8. Deliverable

**normalized_skills.csv**

# DAY 18 — BUSINESS DASHBOARD

## 1. Objective

The objective of Day 18 was to create a business dashboard for analyzing job and skill demand using Power BI.

## 2. Tool Used

- Microsoft Power BI

## 3. Dataset Used

**normalized_skills.csv**

Final dataset:

**14,146 rows × 24columns**

## 4. KPI Analysis

### Total Jobs

**14,146**

### Unique Skills

**6,396**

### Top Skill

The top skill was identified based on skill frequency.

#### Top Job Role

**Software Engineering**

**Count: 83**

#### Most Required Technology

The most frequently required technology was identified based on the skill count.

#### Top Skill Category

A separate Skill Category field was not available in the dataset, so this KPI was not created.

## 5. Dashboard Charts

#### 5.1 Top 20 Skills

A horizontal bar chart was created to display the top 20 skills.

**Category / Y-axis:** `skills`

**Values / X-axis:** `skills` → Count

#### 5.2 Skill Demand by Job Role

A horizontal bar chart was created to show skill demand by job role.

**Category / Y-axis:** `title`

**Values / X-axis:** `skills` → Count

#### 5.3 Skill Trends

A line chart was created to analyze skill demand using job posting dates.

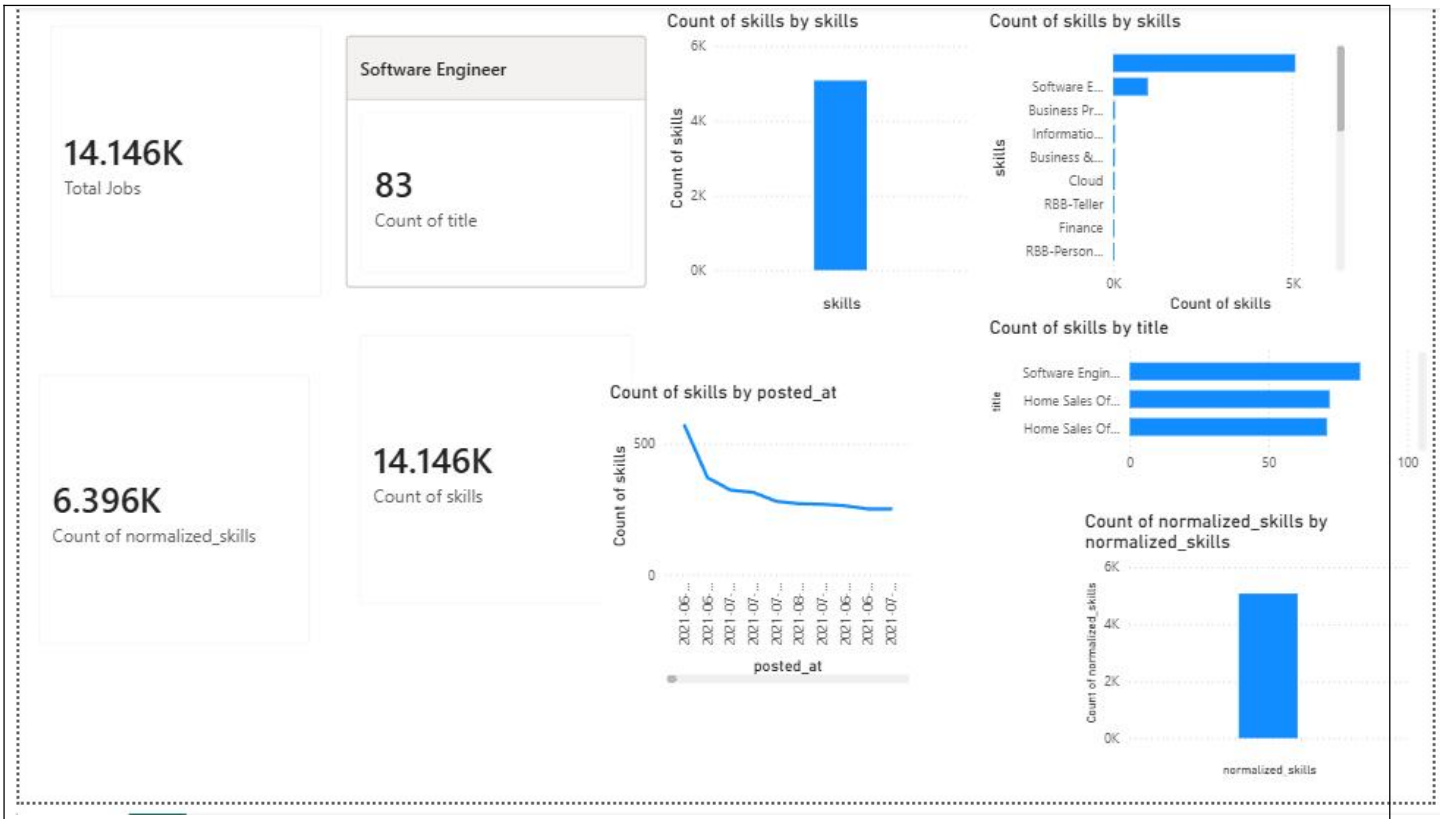
**Category / X-axis:** `posted_at`

**Values / Y-axis:** `skills` → Count

#### 5.4 Skill Category Distribution

This chart was not created because a separate Skill Category field was not available in the dataset.

## 6. Dashboard Screenshot




## 7. Deliverable

**Job_Skill_Analytics.pbix**

# DAY 19 – BUILD AN APPLICATION / API

## 19.1 Objective

The objective of Day 19 was to convert the Automatic Skill Extraction project into a usable application. The application allows the user to enter a job description and detects the required skills. It also maps the detected skills into relevant categories. Streamlit was used to develop the simple web application.

## 19.2 User Input

The application accepts a job description from the user.

**Example Input:**

"Looking for Data Analyst with SQL, Python, Power BI and Excel experience."

## 19.3 Login Page

A login page was added to provide basic access control to the application.

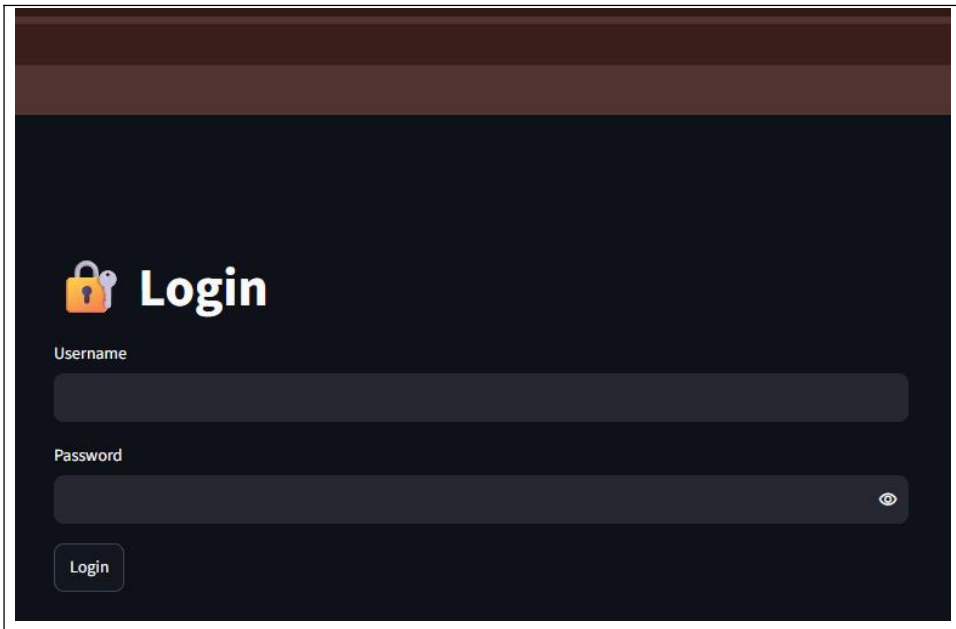
The login page contains:

- Username
- Password
- Login button

**Login Credentials Used for Testing**

Username: `admin`

password: `1234`




## 19.4 Detected Skills

After entering the job description and clicking the **Extract Skills** button, the application detects the skills present in the given job description.

**Detected Skills:**

- Python
- SQL
- Power BI
- Excel

## 19.5 Category Mapping

The detected skills are mapped into their respective categories.

| Category | Skill |
|---|---|
| Programming | Python |
| Database | SQL |
| BI | Power BI |
| Analytics | Excel |

## 19.6 Technology Used

The following technologies were used to develop the application:

- Python
- Streamlit

Streamlit was used to create the web-based application interface.

## 19.7 Application Architecture

The application follows the workflow:

**User → Web Interface → Preprocessing → Skill Extraction → Skill Normalization → Category Mapping → Final Skills**

The user provides a job description through the web interface. The system extracts the required skills and maps them to their respective categories.

## 19.8 Streamlit Application

The main application was developed using `app.py`.

```
import streamlit as stfrom utils.skill_extractor import extract_skillsfrom utils.category_mapping import get_category
st.set_page_config(
  page_title="Skill Extraction",
  page_icon="💼",
  layout="centered"
)
if "logged_in" not in st.session_state:
  st.session_state.logged_in = False
if not st.session_state.logged_in:

  st.title("🔐 Login")

  username = st.text_input("Username")
  password = st.text_input("Password", type="password")

  if st.button("Login"):

    if username == "admin" and password == "1234":
      st.session_state.logged_in = True
      st.success("Login successful!")
      st.rerun()
    else:
      st.error("Invalid username or password")
```

```
else:

  st.title("💼 Automatic Skill Extraction")
  st.write("Paste a job description to detect required skills and categories.")

  if st.button("Logout"):
    st.session_state.logged_in = False
    st.rerun()

  job_description = st.text_area(
    "📝 Paste Job Description",
    placeholder="Looking for Data Analyst with SQL, Python, Power BI and Excel experience.",
    height=180
  )

  if st.button("🔍 Extract Skills"):

    if job_description.strip():

      detected_skills = extract_skills(job_description)

      st.subheader("🎯 Detected Skills")

      if detected_skills:
        st.success(", ".join(detected_skills))

        st.subheader("🗂️ Categories")

        for skill in detected_skills:
          category = get_category(skill)
          st.write(f"**{category}** → {skill}")

      else:
        st.warning("No skills detected.")

    else:
      st.warning("Please paste a job description first.")
```

```
import streamlit as st
from utils.skill_extractor import extract_skills
from utils.category_mapping import get_category

st.set_page_config(
    page_title="Skill Extraction",
    page_icon="💼",
    layout="centered"
)

# Login Page
if "logged_in" not in st.session_state:
    st.session_state.logged_in = False

if not st.session_state.logged_in:

    st.title("🔐 Login")

    username = st.text_input("Username")
    password = st.text_input("Password", type="password")

    if st.button("Login"):

        if username == "admin" and password == "1234":
            st.session_state.logged_in = True
            st.success("Login successful!")
            st.rerun()
        else:
            st.error("Invalid username or password")

# Skill Extraction Page
else:

    st.title("💼 Automatic Skill Extraction")
```

Ln 1, Col 23 | 1,811 characters | Plain text

## 19.9 Skill Extraction

The skill extraction module checks the job description for predefined skills and returns the detected skills.

**File:** `utils/skill_extractor.py`

```
def extract_skills(text):
    skills = ["Python", "SQL", "Power BI", "Excel"]

    detected_skills = []

    for skill in skills:
        if skill.lower() in text.lower():
            detected_skills.append(skill)
```

```
    return detected_skills
```

### 19.10 Category Mapping

The category mapping module assigns a category to each detected skill.

**File:** `utils/category_mapping.py`

```
def get_category(skill):
    categories = {
        "Python": "Programming",
        "SQL": "Database",
        "Power BI": "BI",
        "Excel": "Analytics"
    }

    return categories.get(skill, "Other")
```

### 19.11 Project Structure

The application was organized into separate files and folders.

```
Skill_Extraction_App
|
├── app.py
├── app_backup.py
├── model/
├── data/
├── utils/
|    ├── category_mapping.py
|    └── skill_extractor.py
├── requirements.txt
└── README.md
```

### 19.12 Requirements

The application requires Streamlit.

**requirements.txt**

```
streamlit
```

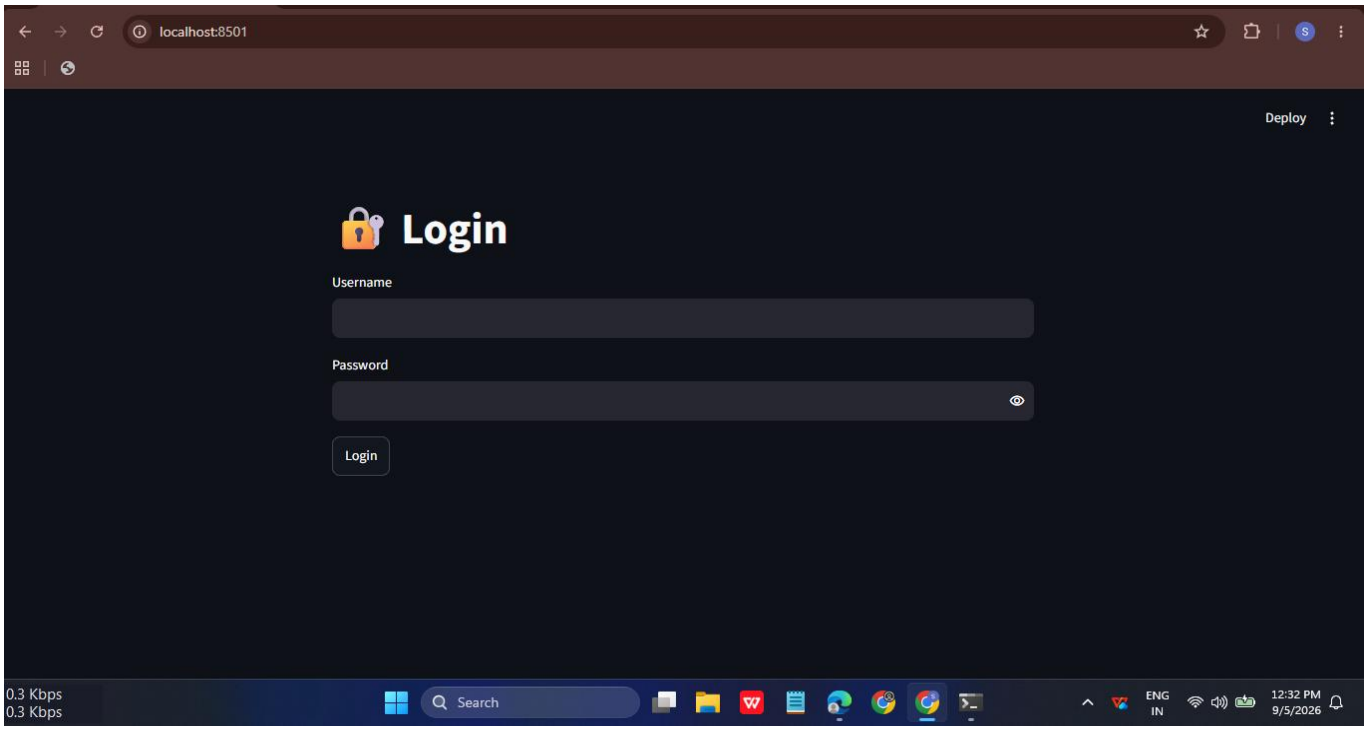
### 19.13 Application Demo

The application was successfully run locally using Streamlit.

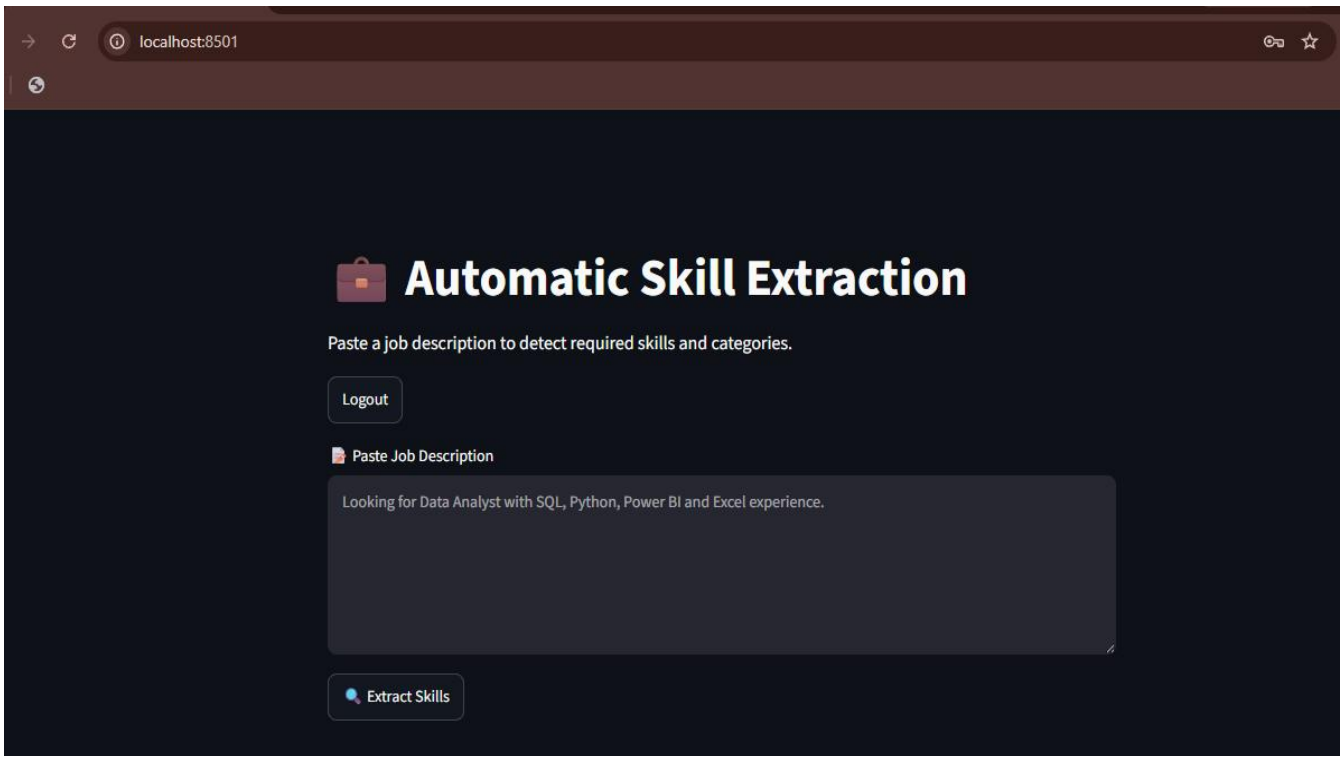
The user first logs in through the Login Page. After successful login, the user can enter a job description and click the **Extract Skills** button.

The application then displays the detected skills and their corresponding categories.

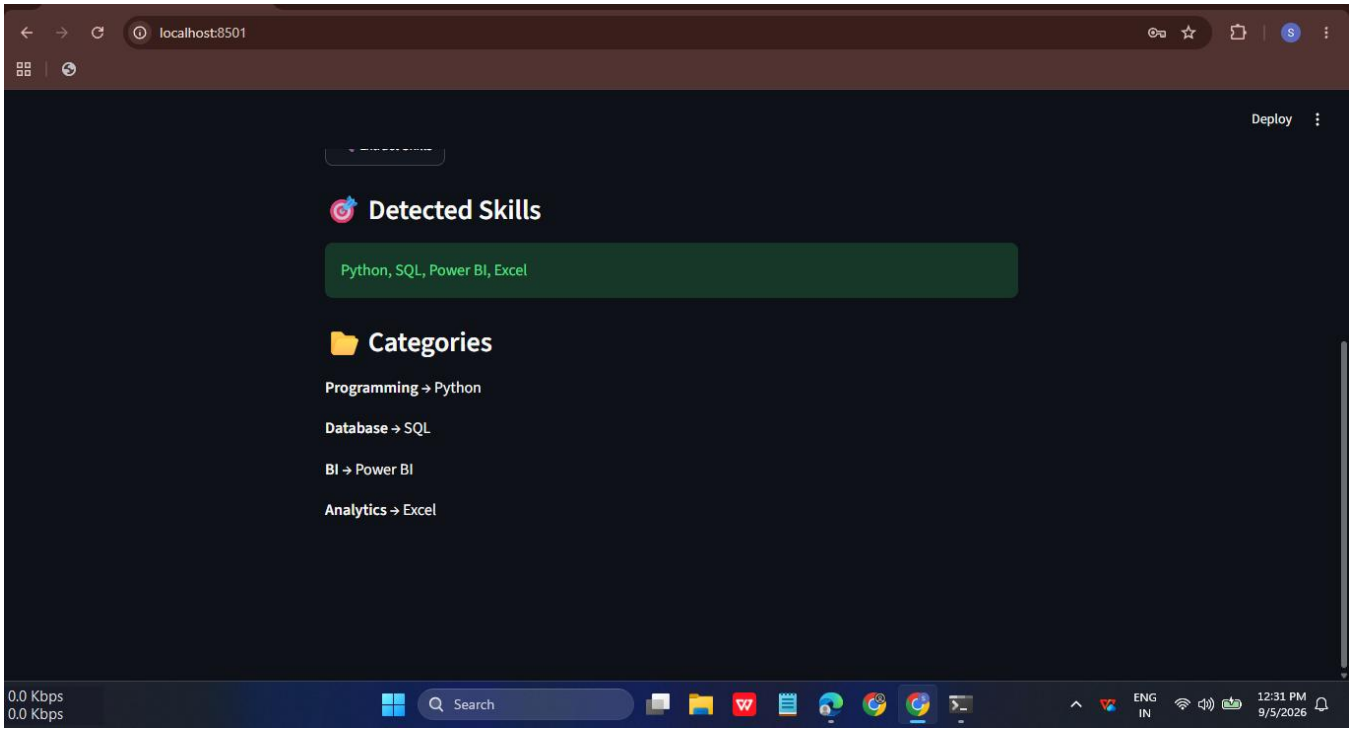
**LOGIN PAGE**



## MAIN APPLICATION PAGE



**FINAL OUTPUT**




## 19.14 Result

The application successfully detected the required skills from the given job description and mapped them into appropriate categories.

**Final Output:**

- Python → Programming
- SQL → Database
- Power BI → BI
- Excel → Analytics

The application was successfully tested locally and produced the expected output.

## 20. Testing & Results

**Testing:**
The application was tested by providing a sample job description as input. The system successfully detected the required skills and displayed their corresponding categories.

**Test Input:**
"Looking for Data Analyst with SQL, Python, Power BI and Excel experience."

**Test Output:**

- Python → Programming
- SQL → Database
- Power BI → BI
- Excel → Analytics

**Result:**
The application successfully produced the expected output for the given job description.

## 21. Challenges & Limitations

During the development of the Automatic Skill Extraction project, some challenges and limitations were observed.

### Challenges

- Handling missing values in the job dataset.
- Cleaning and preprocessing job description text.
- Extracting skills from unstructured job descriptions.
- Handling different skill names and variations.
- Testing the application and ensuring that the expected skills were detected correctly.

### Limitations

- The current application uses a predefined set of skills for extraction.
- Skills that are not included in the predefined list may not be detected.
- The application is currently tested and run locally using Streamlit.
- The current category mapping is based on predefined skill categories.

## 22. Conclusion

The Automatic Skill Extraction from Job Descriptions project was successfully developed using Natural Language Processing and Machine Learning techniques. The project processes job descriptions and identifies relevant skills from the given text.

The project included data collection, data cleaning, NLP preprocessing, skill extraction, TF-IDF and N-Gram analysis, machine learning, model evaluation, skill normalization, categorization, visualization, and application development.

The final Streamlit application successfully accepts a job description, detects the required skills, and maps them into relevant categories.

**Overall, the project successfully demonstrates an automated approach for extracting and categorizing skills from job descriptions.**

## 23. Future Scope

The Automatic Skill Extraction project can be further improved and extended in the future.

- Resume and job description matching can be added.
- Candidate ranking can be developed based on required skills.
- Skill-gap analysis can be implemented.
- A recommendation system can be developed for suitable jobs or candidates.
- The skill database can be expanded to include more technical and non-technical skills.
- The application can be deployed online so that it can be accessed by users remotely.

## 24. Internship Learning Outcomes

During this internship, I developed and improved my technical and analytical skills through practical project work.

- Learned how to collect and understand a real-world job dataset.
- Gained practical knowledge of data cleaning and NLP preprocessing.
- Learned skill extraction using rule-based and NLP techniques.
- Gained understanding of TF-IDF, N-Grams, word embeddings, and semantic similarity.
- Learned about Named Entity Recognition (NER) and Machine Learning techniques.
- Learned how to evaluate and compare models.
- Gained knowledge of skill normalization and categorization.
- Learned to create business dashboards and visualizations.
- Learned how to develop a simple web application using Streamlit.
- Improved practical problem-solving and project-development skills.

## 25. References


- Monster India Job Dataset – Dataset used for the project.
- Python Documentation – Python programming and implementation reference.
- Pandas Documentation – Data handling and analysis.
- NLTK Documentation – Natural Language Processing and text preprocessing.
- Scikit-learn Documentation – Machine Learning and model evaluation.
- Streamlit Documentation – Web application development.
- Google Colab – Development and execution environment.


## 26. Appendix

The appendix contains supporting materials related to the project for reference.

### 26.1 Code Snippets

Important code snippets used during the development of the project are included for reference.

### 26.2 Sample Data

Sample job descriptions and skill-related data used during the project are included for reference.

### 26.3 Additional Results

Additional outputs and results obtained during the project are included for reference.

### 26.4 Application Details

The final Streamlit application files and project structure are included as supporting material.